

ΨΗΦΙΑΚΕΣ ΤΗΛΕΠΙΚΟΙΝΩΝΙΕΣ

Πανεπιστήμιο Πατρών, Τμήμα Μηχανικών Η/Υ και Πληροφορικής
Ακαδημαϊκό Έτος 2023-2024 - 1^ο σετ Εργαστηριακών Ασκήσεων

Παπαχριστοφίλου Σαράντης

ΑΜ 1072600, Έτος: 5^ο

ΜΕΡΟΣ Α΄

Ερώτηση 1

Α) Στο ερώτημα αυτό καλούμαστε να εντοπίσουμε τα σύμβολα της πηγής (parrot.png) και τις πιθανότητες εμφάνισής τους. Γι' αυτό διαβάζουμε την εικόνα μέσω της **imread** και παίρνουμε τα σύμβολα χρησιμοποιώντας την **unique(parrot(:))**. Οι πιθανότητες εμφάνισης των συμβόλων υπολογίζονται από το ζεύγος εντολών: **histogram = histcounts(parrot(:), [sourceSymbols; inf]);** και **probabilities = histogram / numel(parrot);**. Συγκεκριμένα, μετατρέπουμε το 2D μητρώο της εικόνας σε διάνυσμα με μέγεθος όσα είναι τα πίκσελ και υπολογίζουμε το ιστόγραμμα της εικόνας. Για τον υπολογισμό των πιθανοτήτων, διαιρούμε τον αριθμό εμφανίσεων κάθε πίκσελ με τον συνολικό αριθμό πίκσελ στην εικόνα.

```
% Διαβάζουμε την εικόνα
parrot = imread('parrot.png');
% Προβολή της εικόνας
imshow(parrot), title('Original Grayscale Image');

% Εντοπισμός των μοναδικών συμβόλων της εικόνας
sourceSymbols = unique(parrot(:));

% Υπολογισμός Πιθανοτήτων με χρήση της histcounts function
histogram = histcounts(parrot(:), [sourceSymbols; inf]);
probabilities = histogram / numel(parrot);

% Display για κάθε pixel και της πιθανότητάς του
disp('Source Symbols and Probabilities:');
disp(table(double(sourceSymbols), probabilities, 'VariableNames', {'Symbol', 'Probability'}));
```

Αποτελέσματα

```
>> Ena_a
Source Symbols and Probabilities:
   Symbol   Probability
   -----
         0         0.096167
        17         0.0814
        34         0.068333
        51         0.0625
        68         0.076833
        85         0.090467
       102         0.11317
       119         0.0906
       136         0.0965
       153         0.067133
       170         0.0389
       187         0.032933
       204         0.033667
       221         0.0259
       238         0.022367
       255         0.0031333
```

B) Στο ερώτημα αυτό, για την κωδικοποίηση Huffman αξιοποιούμε την συνάρτηση `huffmandict` για να δημιουργήσουμε το Huffman dictionary, βάσει των συμβόλων που βρήκαμε στο ερώτημα 1α και των πιθανοτήτων τους. Η εντροπία υπολογίζεται χρησιμοποιώντας τον τύπο $H = -\sum p_i \cdot \log_2(p_i)$, το μέσο μήκος κώδικα υπολογίζεται πολλαπλασιάζοντας το άθροισμα των πιθανοτήτων με το μήκος της κωδικοποίησης Huffman για κάθε σύμβολο και η αποδοτικότητα του κώδικα ως ο λόγος της εντροπίας προς το μέσο μήκος κώδικα.

```
% Δημιουργία dictionary huffman
htree = huffmandict(sourceSymbols, probabilities);
% Υπολογισμός Entropy(H), Mean Code Length(L), Efficiency(E)
H = -sum(probabilities .* log2(probabilities));
L = sum(probabilities .* cellfun('length', htree(:, 2)))/length(sourceSymbols);
%L = mean(cellfun('length', htree(:, 2)));
E = H / L;

% Αποτελέσματα
disp(['Entropy (H): ' num2str(H)]);
disp(['Mean Code Length (L): ' num2str(L)]);
disp(['Efficiency (E): ' num2str(E)]);
```

Αποτελέσματα

```
Entropy (H): 3.7831
Mean Code Length (L): 3.8374
Efficiency (E): 0.98585
```

Σχόλιο

Εντροπία: Κάθε Σύμβολο αναπαρίσταται κατά μέσο όρο από πληροφορία ίση με 3.7831bits

Μέσο Μήκος Κώδικα: Είναι λίγο μεγαλύτερο από την εντροπία, συνεπώς, η κωδικοποίηση Huffman είναι ελάχιστα λιγότερο αποδοτική από το θεωρητικό ελάχιστο.

Αποδοτικότητα: Πολύ υψηλή αποδοτικότητα που υποδηλώνει ότι η κωδικοποίηση Huffman ήταν αρκετά αποτελεσματική. Αυτό εξαρτάται από τα χαρακτηριστικά της πηγής.

Ερώτηση 2

A) Για την θεώρηση της δεύτερης τάξης επέκτασης πηγής, διαβάζουμε τα πίξελ της εικόνας μέσω δύο for loops και δημιουργούμε ένα τρισδιάστατο μητρώο, το οποίο αποτελείται από ζεύγη συμβόλων και κάθε ζεύγος αντιστοιχεί σε δύο διαδοχικά πίξελ στην εικόνα. Στη συνέχεια, το μητρώο **symbolPairs** αναδιαμορφώνεται σε ένα δισδιάστατο μητρώο (**reshapedPairs**) για ευκολότερη επεξεργασία. Με τη χρήση της συνάρτησης **unique**, βρίσκονται οι μοναδικές γραμμές στο **reshapedPairs**. Στη συνέχεια, υπολογίζονται οι πιθανότητες εμφάνισης για κάθε ζεύγος συμβόλων δεύτερης τάξης.

```

% Διάβασμα Εικόνας
grayImage = imread('parrot.png');

% Διατρέχουμε τα πίξελ της εικόνας και δημιουργούμε μητρώο
[rows, cols] = size(grayImage);
symbolPairs = zeros(rows - 1, cols - 1, 2);

for i = 1:(rows - 1)
    for j = 1:(cols - 1)
        symbolPairs(i, j, 1) = grayImage(i, j);
        symbolPairs(i, j, 2) = grayImage(i + 1, j + 1);
    end
end

% Reshape το μητρώο για εύκολη επεξεργασία
reshapedPairs = reshape(symbolPairs, [], 2);

% Μοναδικές Γραμμές
[uniquePairs, ~, idx] = unique(reshapedPairs, 'rows');
pairCounts = histcounts(idx, [unique(idx); max(idx) + 1]);
pairProbabilities = pairCounts / numel(idx);

% Display Μοναδικών Συμβόλων και των Πιθανοτήτων τους
disp('Second-Order Source Symbol Pairs and Probabilities:');
disp(table(double(uniquePairs), pairProbabilities', 'VariableNames', {'SymbolPair', 'Probability'}));

```

Αποτελέσματα

Second-Order Source Symbol Pairs			34	0	0.0022259	68	0	0.00047216
SymbolPair	Probability		34	17	0.016121	68	17	0.0016863
			34	34	0.027857	68	34	0.0039796
			34	51	0.011871	68	51	0.01349
0	0	0.074837	34	68	0.0040808	68	68	0.031331
0	17	0.015008	34	85	0.0020235	68	85	0.015244
0	34	0.0029679	34	102	0.001349	68	102	0.0048902
0	51	0.00097804	34	119	0.00057334	68	119	0.0022596
0	68	0.00074196	34	136	0.00064079	68	136	0.0011804
0	85	0.00030353	34	153	0.00026981	68	153	0.00067451
0	102	6.7451e-05	34	170	0.00020235	68	170	0.00050589
0	119	0.0001349	34	187	0.00030353	68	187	0.00023608
0	136	6.7451e-05	34	204	0.00033726	68	204	0.00037098
0	153	0.0001349	34	221	0.0001349	68	221	0.00016863
0	170	3.3726e-05	34	238	6.7451e-05	68	238	0.00016863
0	204	6.7451e-05	34	255	3.3726e-05	68	255	3.3726e-05
0	221	3.3726e-05	51	0	0.00067451	85	0	0.0001349
17	0	0.015008	51	17	0.0051938	85	17	0.00097804
17	17	0.040437	51	34	0.01322	85	34	0.0020235
17	34	0.01575	51	51	0.021315	85	51	0.00516
17	51	0.0043169	51	68	0.01231	85	68	0.014266
17	68	0.0024282	51	85	0.0036761	85	85	0.03757
17	85	0.0010792	51	102	0.0021584	85	102	0.020235
17	102	0.00080942	51	119	0.0011467	85	119	0.0051263
17	119	0.00040471	51	136	0.00074196	85	136	0.0021922
17	136	0.00030353	51	153	0.00040471	85	153	0.0011804
17	153	0.00040471	51	170	0.00010118	85	170	0.00067451
17	170	0.00023608	51	187	0.00026981	85	187	0.00037098
17	187	0.00020235	51	204	0.0001349	85	204	0.00023608
17	204	3.3726e-05	51	221	0.00016863	85	221	0.00016863
			51	238	0.00010118	85	238	3.3726e-05
			51	255	3.3726e-05	85	255	3.3726e-05

102	0	6.7451e-05	136	0	3.3726e-05	170	17	3.3726e-05
102	17	0.00060706	136	17	0.00023608	170	51	0.00020235
102	34	0.00091059	136	34	0.00023608	170	68	0.00033726
102	51	0.0021247	136	51	0.0010455	170	85	0.00064079
102	68	0.0069138	136	68	0.0016863	170	102	0.00087687
102	85	0.019122	136	85	0.0035749	170	119	0.0015851
102	102	0.055715	136	102	0.0065091	170	136	0.0040134
102	119	0.019493	136	119	0.01548	170	153	0.0098816
102	136	0.0048902	136	136	0.045058	170	170	0.012074
102	153	0.0017537	136	153	0.016526	170	187	0.0065091
102	170	0.00080942	136	170	0.0034737	170	204	0.0024957
102	187	0.00060706	136	187	0.0014839	170	221	0.00026981
102	204	0.00020235	136	204	0.00070824	170	238	0.00020235
102	221	0.00030353	136	221	6.7451e-05	170	255	3.3726e-05
102	238	3.3726e-05	136	238	0.0001349	187	51	6.7451e-05
102	255	3.3726e-05	136	255	0.00016863	187	68	6.7451e-05
119	0	6.7451e-05	153	17	0.00010118	187	85	0.0001349
119	17	0.00037098	153	34	6.7451e-05	187	102	0.00023608
119	34	0.00077569	153	51	0.00043843	187	119	0.00053961
119	51	0.0011129	153	68	0.00050589	187	136	0.0010792
119	68	0.0025969	153	85	0.0012816	187	153	0.0036761
119	85	0.0062392	153	102	0.002833	187	170	0.0075883
119	102	0.018144	153	119	0.0065091	187	187	0.01032
119	119	0.037604	153	136	0.018954	187	204	0.0066102
119	136	0.016762	153	153	0.024586	187	221	0.0021922
119	153	0.0045192	153	170	0.0082628	187	238	0.00053961
119	170	0.0013153	153	187	0.0025632	187	255	0.00010118
119	187	0.00060706	153	204	0.0010118			
119	204	0.00020235	153	221	0.00016863			
119	221	0.00030353	153	238	3.3726e-05			
119	238	0.0001349	153	255	0.00010118			
119	255	3.3726e-05						

204	51	6.7451e-05	238	68	6.7451e-05
204	68	0.00010118	238	119	6.7451e-05
204	85	0.00020235	238	153	0.00047216
204	102	3.3726e-05	238	170	0.00040471
204	119	0.00026981	238	187	0.00023608
204	136	0.00057334	238	204	0.00094432
204	153	0.0017875	238	221	0.0057334
204	170	0.0026306	238	238	0.013457
204	187	0.00688	238	255	0.0012478
204	204	0.013962	255	170	3.3726e-05
204	221	0.0056659	255	187	6.7451e-05
204	238	0.0015177	255	204	6.7451e-05
204	255	0.00020235	255	221	0.00020235
221	34	3.3726e-05	255	238	0.0021247
221	85	3.3726e-05	255	255	0.00067451
221	102	3.3726e-05			
221	119	6.7451e-05			
221	136	0.00033726			
221	153	0.00091059			
221	170	0.00094432			
221	187	0.0022933			
221	204	0.0064416			
221	221	0.01059			
221	238	0.0040471			
221	255	0.00043843			

Β) Ο παρακάτω κώδικας μετατρέπει τα μοναδικά ζεύγη σε ένα κελί (**symbolPairsCell**) για να προετοιμάσει τα δεδομένα για τον κώδικα Huffman. Κάθε κελί περιέχει ένα ζεύγος συμβόλων. Στη συνέχεια, κατασκευάζουμε το λεξικό Huffman χρησιμοποιώντας τα κελιά των ζευγών συμβόλων και των πιθανοτήτων τους. Η εντροπία υπολογίζεται χρησιμοποιώντας τον τύπο $H = -\sum p_i \cdot \log_2(p_i)$, το μέσο μήκος κώδικα υπολογίζεται πολλαπλασιάζοντας το άθροισμα των πιθανοτήτων με το μήκος της κωδικοποίησης Huffman για κάθε σύμβολο και η αποδοτικότητα του κώδικα ως ο λόγος της εντροπίας προς το μέσο μήκος κώδικα.

```
symbolPairsCell = mat2cell(uniquePairs, ones(size(uniquePairs, 1), 1), 2);

% Δημιουργία Huffman Dictionary για την δεύτερηςτάξης επέκταση
dictPairs = huffmandict(symbolPairsCell, pairProbabilities);

% Υπολογισμός Entropy(H), Mean Code Length(L), Efficiency(E)
HPairs = -sum(pairProbabilities .* log2(pairProbabilities));
%LPairs = mean(cellfun('length', dictPairs(:, 2)));
LPairs = sum(pairProbabilities .* cellfun('length', dictPairs(:, 2)));
EPairs = HPairs / LPairs;

% Αποτελέσματα
disp(['Entropy (H) for second-order extension: ' num2str(HPairs)]);
disp(['Mean Code Length (L) for second-order extension: ' num2str(LPairs)]);
disp(['Efficiency (E) for second-order extension: ' num2str(EPairs)]);
```

Αποτελέσματα

```
Entropy (H) for second-order extension: 6.0474
Mean Code Length (L) for second-order extension: 6.0739
Efficiency (E) for second-order extension: 0.99563
```

Σχόλιο

Εντροπία: Κάθε Ζεύγος Συμβόλων αναπαρίσταται κατά μέσο όρο από πληροφορία ίση με 6.0474bits

Μέσο Μήκος Κώδικα: Κάθε Ζεύγος Συμβόλων αναπαρίσταται κατά μέσο όρο με 6.0739bits

Αποδοτικότητα: Σχεδόν τέλεια αποδοτικότητα που υποδηλώνει ότι η κωδικοποίηση Huffman ήταν αρκετά αποτελεσματική. Αυτό εξαρτάται από τα χαρακτηριστικά της πηγής.

Γ) **Εντροπία (H):** Η εντροπία αυξάνεται σημαντικά στη δεύτερη επέκταση, καθώς προστίθενται ζεύγη συμβόλων. Αυτό υποδηλώνει αυξημένη πολυπλοκότητα στη δεύτερη τάξη της πηγής.

Μέσο Μήκος Κώδικα (L): Το μέσο μήκος του κώδικα αυξάνεται επίσης, προσαρμοζόμενο στη νέα πολυπλοκότητα της δεύτερης επέκτασης.

Αποδοτικότητα (E): Η αποτελεσματικότητα παραμένει υψηλή στη δεύτερη επέκταση, υποδεικνύοντας ότι ο κώδικας είναι ακόμη αποτελεσματικός στη συμπίεση των δεδομένων.

Συνεπώς, η δεύτερης τάξης επέκταση πηγής εισάγει περισσότερη πολυπλοκότητα, κάνοντας πιο δύσκολη την συμπίεσή της. Τα σύμβολα αυξάνονται σε μεγάλο βαθμό, όπως και το μέσο μήκος κώδικα. Ωστόσο, η κωδικοποίηση Huffman είναι και στις δύο περιπτώσεις αρκετά αποδοτική και άρα μπορούν να επιλεγούν και οι δύο.

Ερώτηση 3

A) Ο τύπος $H(X^2) = 2H(X)$ για τον υπολογισμό της εντροπίας της δεύτερης τάξης επέκτασης πηγής δεν ισχύει στην συγκεκριμένη περίπτωση. Αυτό συμβαίνει καθώς ο τύπος ισχύει όταν οι πιθανότητες εμφάνισης των συμβόλων είναι ανεξάρτητες μεταξύ τους. Στην προκειμένη περίπτωση αυτό δεν ισχύει, αφού στην επέκταση δεύτερης τάξης υπολογίζουμε την από κοινού πιθανότητα συμβόλων, δημιουργώντας ζευγάρια. Άρα, εντροπία δεύτερης τάξης επέκτασης πηγής, αφορά το περιεχόμενο πληροφοριών των ζευγών συμβόλων και τις αλληλεπιδράσεις μεταξύ αυτών των ζευγών. Δεν σχετίζεται άμεσα με το τετράγωνο της εντροπίας των ατομικών συμβόλων.

B) Για τον υπολογισμό φράγματος της μορφής $a \leq L < b$, χρησιμοποιήσαμε τον τύπο $H \leq L < H+1$.

Άρα, για το ερώτημα 1 τα φράγματα που υπολογίσαμε είναι:

Constraint: $3.7831 < L < 4.7831$

Και για το ερώτημα 2 είναι:

Constraint: $6.0474 < L < 7.0474$

Ερώτηση 4

Για το ερώτημα αυτό κωδικοποιούμε και αποκωδικοποιούμε την εικόνα με την χρήση του Dictionary Huffman. Υπολογίζουμε τα bits αναπαράστασης της εικόνας για την δυαδική της αναπαράστασης και για την κωδικοποίηση Huffman. Και ο λόγος αυτών των bits είναι ο ζητούμενος.

```
% Κωδικοποιημένη Εικόνα
encodedImage = huffmanenco(parrot(:), htree);

% Αποκωδικοποιημένη Εικόνα
decodedImage = huffmandeco(encodedImage, htree);
decodedImage = reshape(decodedImage, size(parrot));

% Δυαδική Αναπαράσταση Εικόνας
binaryImage = reshape((dec2bin(typecast(parrot(:), 'uint8'), 8) - '0').', 1, []);

% Αριθμός bits της δυαδικής αναπαράστασης
binaryRepresentationBits = numel(binaryImage);

% Αριθμός bits της κωδικοποίησης Huffman
bits_huffman = numel(encodedImage);

% Λόγος Συμπίεσης
compressionRatio = bits_huffman / binaryRepresentationBits;

% Display Λόγου Συμπίεσης
disp(['J = #bits Huffman coding / #bits binary representation ] = ' num2str(compressionRatio)]);
```

Αποτελέσματα

J = #bits Huffman coding / #bits binary representation] = 0.47967

Ερώτηση 5

Για τον υπολογισμό των ζητούμενων στο ερώτημα αυτό δημιουργούμε τις αντιστοιχίες συναρτήσεων. Πιο συγκεκριμένα, για την παράμετρο p , δημιουργείται η `estimate_p`, για την χωρητικότητα η `capacity` και για την αμοιβαία πληροφορία η έτοιμη συνάρτηση `mutualInfo()` που περιλαμβάνεται στο πακέτο `Image mutual Information`.

```
% Θέτουμε X την κωδικοποιημένη εικόνα
X = encodedImage;

% Καλούμε την έτοιμη συνάρτηση που μας έχει δοθεί
Y = binary_symmetric_channel(X);

% Υπολογισμός Παραμέτρου p
p_estimate = estimate_p(X, Y);

% Προβολή παραμέτρου p
disp(['Estimated p: ' num2str(p_estimate, '%.2f')]);

% Υπολογισμός Χωρητικότητας Καναλιού
capacity = capacity(p_estimate);

% Προβολή Χωρητικότητας Καναλιού
disp(['Channel Capacity: ' num2str(capacity, '%.2f')]);

% Αμοιβαία Πληροφορία
I = mutualInfo(X, Y);
disp(['Mutual Information: ' num2str(I, '%.2f')]);

function capacity = capacity(p_estimate)
    H_p = -p_estimate * log2(p_estimate) - (1 - p_estimate) * log2(1 - p_estimate);

    capacity = 1 - H_p;
end

function p_estimate = estimate_p(x, y)
    num_errors = sum(x ~= y);

    p_estimate = num_errors / length(x);
end
```

Αποτελέσματα

```
Estimated p: 0.12
Channel Capacity: 0.47
Mutual Information: 0.47
```

ΜΕΡΟΣ Β΄

Ερώτηση 1

Για την υλοποίηση του ζητούμενου συστήματος δημιουργήσαμε τρεις συναρτήσεις. Μία για τον κωδικοποιητή, μία για τον αποκωδικοποιητή και άλλη μία για τον κβαντιστή, σύμφωνα με την εκφώνηση.

Κωδικοποιητής

```
function [ a_q, yh, y ] = dpcm_encoder(x, p, qBits, qMin, qMax)

N = length(x);
r = zeros(p, 1);
R = zeros(p, p);

for i = 1:p
    r(i) = 1/(N - p) * (x(p+1:N)' * x(p+1-i:N-i));
    for j = 1:p
        R(i, j) = 1/(N - p) * (x(p+1-j:N-j)' * x(p+1-i:N-i));
    end
end

a_q = kvantistis(R\r, 8, -2, 2);

y = zeros(N, 1);
yh = zeros(N, 1);

y(1:p) = x(1:p);
yh(1:p) = kvantistis(y(1:p), qBits, qMin, qMax);

yht = yh(p:-1:1);

for i = p+1:N
    yt = a_q' * yht;
    y(i) = x(i) - yt;
    yh(i) = kvantistis(y(i), qBits, qMin, qMax);
    yht = [yt + yh(i); yht(1:p-1)];
end
end
```

Αποκωδικοποιητής

```
function [ yht ] = dpcm_decoder(a_q, yh)

N = length(yh);
p = length(a_q);

yht = zeros(N, 1);
yht(1:p) = yh(1:p);

for i = p+1:N
    yt = a_q' * yht(i-1:-1:i-p);
    yht(i) = yt + yh(i);
end
end
```

Κβαντιστής

```
function [ y_h ] = kvantistis(y, N, min_value, max_value)

delta = abs(max_value - min_value) / 2^N;

if y > max_value
    y = max_value;
elseif y < min_value
    y = min_value;
end

y_h = delta * (floor(y/delta) + 0.5);
end
```

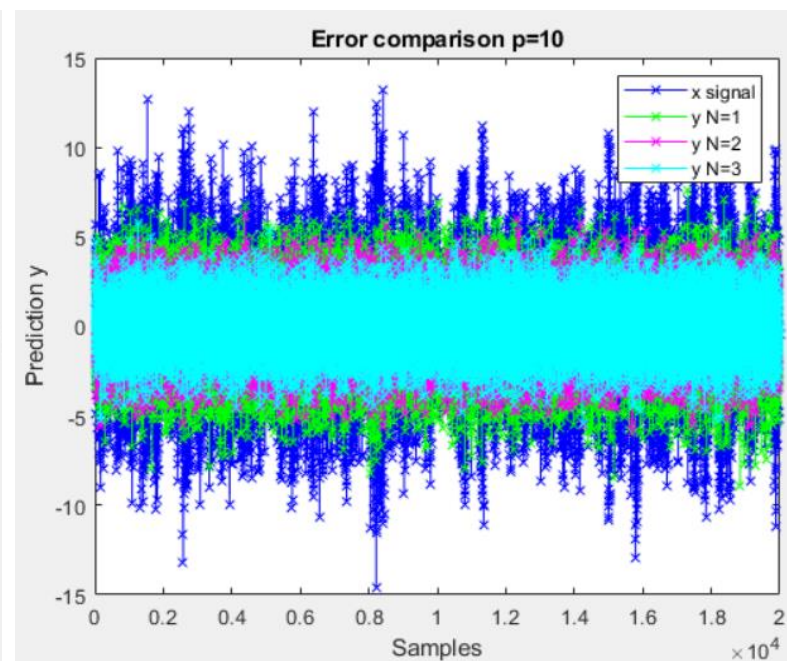
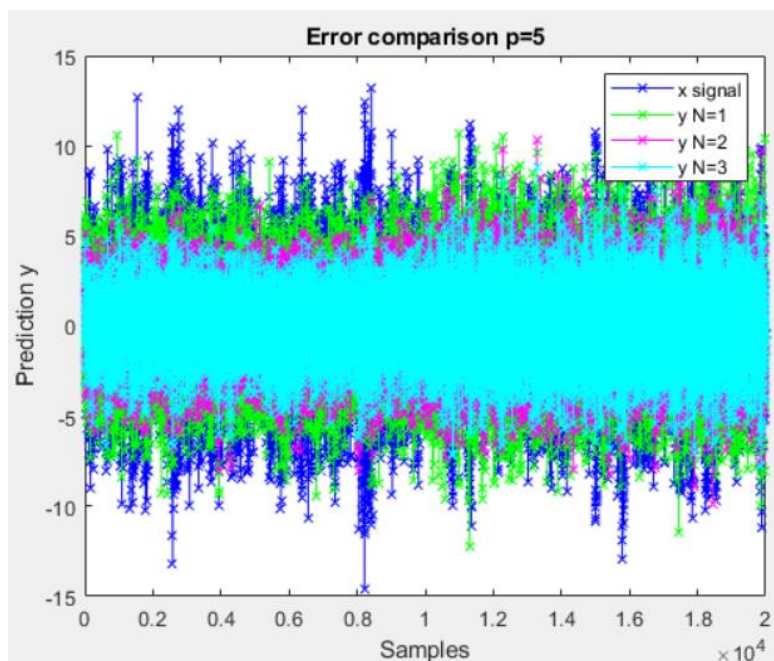
Ερώτηση 2

```
% Φόρτωση source.mat
load('source.mat');
N = length(t);

% Όρια κβάντισης
minVal = -3.5;
maxVal = 3.5;

colors = ['-xb'; '-xg'; '-xm'; '-xc'];

% Ερώτηση 2
r=(1:N);
for p = 5:5:10
    figure
    plot(r, t(r), colors(1,:));
    for i = 1:3
        [~, ~, y] = dpcm_encoder(t, p, i, minVal, maxVal);
        hold on
        plot(r, y(r), colors(1+i,:));
    end
    title(['Error comparison p=', num2str(p)]);
    xlabel('Samples'); ylabel('Prediction y');
    legend('x signal', 'y N=1', 'y N=2', 'y N=3');
end
```



Παρατηρήσεις

Παρατηρούμε ότι εάν αυξησουμε την τάξη p , τότε το σφάλμα πρόβλεψης y μειώνεται.

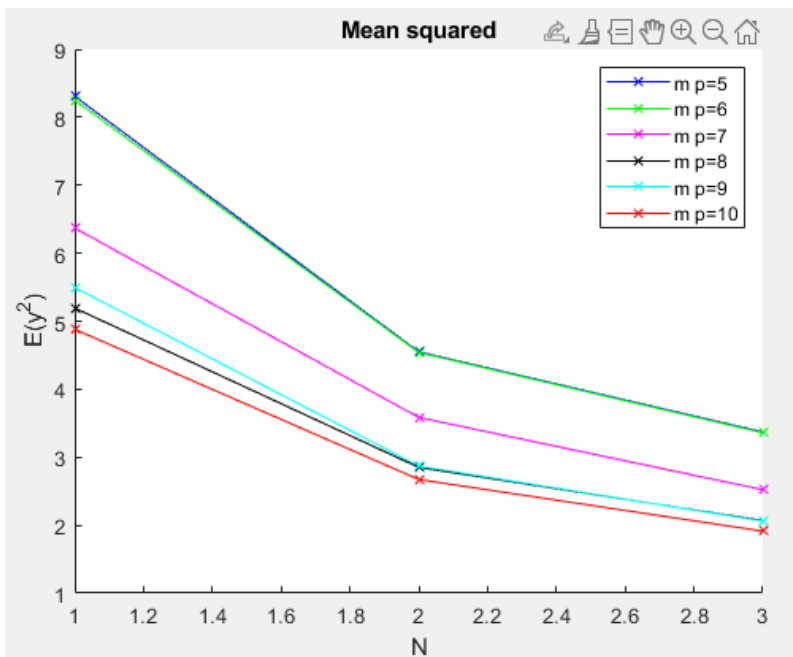
Ερώτηση 3

```
% Φόρτωση source.mat
load('source.mat');
N = length(t);

% Όρια κβάντισης
minVal = -3.5;
maxVal = 3.5;

colors = ['-xb'; '-xg'; '-xm'; '-xk'; '-xc'; '-xr'];

% Ερώτηση 3
r=(1:3);
m = zeros(3, 1);
figure
for p = 5:10
    for i = 1:3
        [a_q, ~, y] = dpcm_encoder(t, p, i, minVal, maxVal);
        m(i) = mean(y.^2);
    end
    hold on
    plot(r, m(r), colors(p-4,:));
    disp(['p: ', num2str(p), ', aq: ', num2str(a_q)])
end
title('Mean squared');
xlabel('N'); ylabel('E(y^2)');
legend('m p=5', 'm p=6', 'm p=7', 'm p=8', 'm p=9', 'm p=10');
```



Τιμές Συντελεστών Προβλέπτη

p: 5, aq: 1.2891	-1.5859	0.99219	-0.53906	-0.023438			
p: 6, aq: 1.2891	-1.5703	0.94531	-0.47656	-0.085938	0.039062		
p: 7, aq: 1.2578	-1.5234	1.1953	-0.96094	0.71094	-0.60156	0.50781	
p: 8, aq: 1.0859	-1.3047	0.94531	-0.61719	0.28906	-0.070312	0.054688	0.35156
p: 9, aq: 1.1484	-1.3047	0.92969	-0.57031	0.19531	0.070312	-0.14844	0.52344
						-0.16406	
p: 10, aq: 1.1016	-1.1797	0.89844	-0.55469	0.24219	-0.054688	0.070312	0.22656
						0.10156	-0.22656

Όσο αυξάνεται η τάξη p , τόσο αυξάνεται ο αριθμός των συντελεστών, δηλαδή χρησιμοποιούνται όλο και περισσότερες τιμές για την πρόβλεψη της επόμενης τιμής στο σήμα.

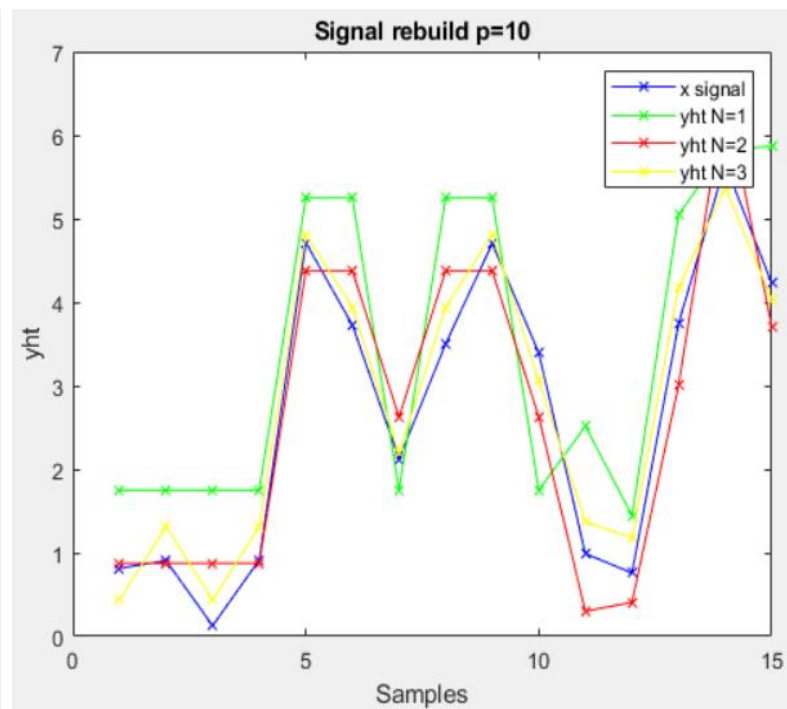
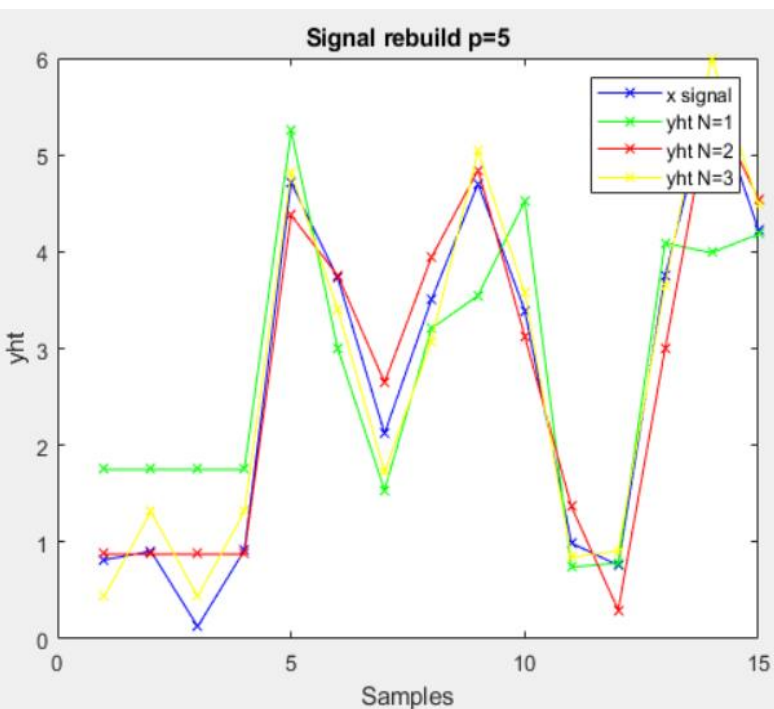
Ερώτηση 4

```
% Φόρτωση source.mat
load('source.mat');
N = length(t);

% Όρια κβάντισης
minVal = -3.5;
maxVal = 3.5;

colors = ['-xb'; '-xg'; '-xr'; '-xy'];

% Ερώτηση 4
r = (1:15);
for p = 5:5:10
    figure
    plot(r, t(r), colors(1,:));
    for i = 1:3
        [a_q, yh, ~] = dpcm_encoder(t, p, i, minVal, maxVal);
        yht = dpcm_decoder(a_q, yh);
        hold on
        plot(r, yht(r), colors(1+i,:));
    end
    title(['Signal rebuild p=', num2str(p)]);
    xlabel('Samples'); ylabel('yht');
    legend('x signal', 'yht N=1', 'yht N=2', 'yht N=3');
end
```



Παρατηρήσεις

Βλέπουμε πως είτε για $p=5$, είτε για $p=10$, όταν χρησιμοποιούνται 3 bits για την ανακτασκευή του σήματος, τότε το ανακατασκευασμένο σήμα προσεγγίζει καλύτερα το αρχικό.

Όσον αφορά το p , όσο αυξάνεται τόσες περισσότερες προηγούμενες τιμές αξιοποιούνται για την πρόβλεψη του επόμενου δείγματος. Άρα, ταυτόχρονα με αυτό αυξάνεται και η ακρίβεια της ανακατασκευής.

ΚΩΔΙΚΑΣ ΜΕΡΟΣ Α

Ερώτημα 1 (Ena_a.m)

```
% Διαβάζουμε την εικόνα
parrot = imread('parrot.png');
% Προβολή της εικόνας
imshow(parrot), title('Original Grayscale Image');

% Εντοπισμός των μοναδικών συμβόλων της εικόνας
sourceSymbols = unique(parrot(:));

% Υπολογισμός Πιθανοτήτων με χρήση της histcounts function
histogram = histcounts(parrot(:), [sourceSymbols; inf]);
probabilities = histogram / numel(parrot);

% Display για κάθε pixel και της πιθανότητάς του
disp('Source Symbols and Probabilities:');
disp(table(double(sourceSymbols), probabilities', 'VariableNames', {'Symbol',
'Probability'}));
```

Ερώτημα 1 (Ena_b.m)

```
% Δημιουργία dictionary huffman
htree = huffmandict(sourceSymbols, probabilities);

% Υπολογισμός Entropy(H), Mean Code Length(L), Efficiency(E)
H = -sum(probabilities .* log2(probabilities));
L = sum(probabilities .* cellfun('length', htree(:, 2)'));
%L = mean(cellfun('length', htree(:, 2)));
E = H / L;

% Αποτελέσματα
disp(['Entropy (H): ' num2str(H)]);
disp(['Mean Code Length (L): ' num2str(L)]);
disp(['Efficiency (E): ' num2str(E)]);
```

Ερώτημα 2 (Duo_A.m)

```
% Διάβασμα Εικόνας
grayImage = imread('parrot.png');

% Διατρέχουμε τα πίξελ της εικόνας και δημιουργούμε μητρώο
[rows, cols] = size(grayImage);
symbolPairs = zeros(rows - 1, cols - 1, 2);

for i = 1:(rows - 1)
    for j = 1:(cols - 1)
        symbolPairs(i, j, 1) = grayImage(i, j);
        symbolPairs(i, j, 2) = grayImage(i + 1, j + 1);
    end
end

% Reshape το μητρώο για εύκολη επεξεργασία
reshapedPairs = reshape(symbolPairs, [], 2);

% Μοναδικές Γραμμές
[uniquePairs, ~, idx] = unique(reshapedPairs, 'rows');
pairCounts = histcounts(idx, [unique(idx); max(idx) + 1]);
pairProbabilities = pairCounts / numel(idx);

% Display Μοναδικών Συμβόλων και των Πιθανοτήτων τους
disp('Second-Order Source Symbol Pairs and Probabilities:');
disp(table(double(uniquePairs), pairProbabilities', 'VariableNames', {'SymbolPair',
'Probability'}));
```

Ερώτημα 2 (Duo_B.m)

```
symbolPairsCell = mat2cell(uniquePairs, ones(size(uniquePairs, 1), 1), 2);

% Δημιουργία Huffman Dictionary για την δεύτερηςτάξης επέκταση
dictPairs = huffmandict(symbolPairsCell, pairProbabilities);

% Υπολογισμός Entropy(H), Mean Code Length(L), Efficiency(E)
HPairs = -sum(pairProbabilities .* log2(pairProbabilities));
%LPairs = mean(cellfun('length', dictPairs(:, 2)));
LPairs = sum(pairProbabilities .* cellfun('length', dictPairs(:, 2)));
EPairs = HPairs / LPairs;

% Αποτελέσματα
disp(['Entropy (H) for second-order extension: ' num2str(HPairs)]);
disp(['Mean Code Length (L) for second-order extension: ' num2str(LPairs)]);
disp(['Efficiency (E) for second-order extension: ' num2str(EPairs)]);
```

Ερώτημα 3 (Tria_B.m)

```
%Fragmata
disp(['Constraint: ' num2str(H) ' <  $\lceil$  < ' num2str(H+1)]);

%Fragmata
disp(['Constraint: ' num2str(HPairs) ' <  $\lceil$  < ' num2str(HPairs+1)]);
```

Ερώτημα 4 (Tessera.m)

```
% Κωδικοποιημένη Εικόνα
encodedImage = huffmanenco(parrot(:), htree);
% Αποκωδικοποιημένη Εικόνα
decodedImage = huffmandeco(encodedImage, htree);
decodedImage = reshape(decodedImage, size(parrot));

% Δυαδική Αναπαράσταση Εικόνας
binaryImage = reshape((dec2bin(typecast(parrot(:), 'uint8'), 8) - '0').', 1, []);

% Αριθμός bits της δυαδικής αναπαράστασης
binaryRepresentationBits = numel(binaryImage);

% Αριθμός bits της κωδικοποίησης Huffman
bits_huffman = numel(encodedImage);

% Λόγος Συμπίεσης
compressionRatio = bits_huffman / binaryRepresentationBits;

% Display Λόγου Συμπίεσης
disp(['J = #bits Huffman coding / #bits binary representation ) = '
num2str(compressionRatio)]);
```

Ερώτημα 5 (Pente.m)

```
% Θέτουμε X την κωδικοποιημένη εικόνα
X = encodedImage;
% Καλούμε την έτοιμη συνάρτηση που μας έχει δοθεί
Y = binary_symmetric_channel(X);
% Υπολογισμός Παραμέτρου p
p_estimate = estimate_p(X, Y);
% Προβολή παραμέτρου p
disp(['Estimated p: ' num2str(p_estimate, '%.2f')]);
% Υπολογισμός Χωρητικότητας Καναλιού
capacity = capacity(p_estimate);
% Προβολή Χωρητικότητας Καναλιού
disp(['Channel Capacity: ' num2str(capacity, '%.2f')]);

% Αμοιβαία Πληροφορία
I = mutualInfo(X, Y);
disp(['Mutual Information: ' num2str(I, '%.2f')]);
```

Συνάρτηση – capacity.m

```
function capacity = capacity(p_estimate)

    H_p = -p_estimate * log2(p_estimate) - (1 - p_estimate) * log2(1 - p_estimate);

    capacity = 1 - H_p;
end
```

Συνάρτηση – estimate_p.m

```
function p_estimate = estimate_p(x, y)
    num_errors = sum(x ~= y);

    p_estimate = num_errors / length(x);
end
```

ΚΩΔΙΚΑΣ ΜΕΡΟΣ Β

dpcm_encoder.m

```
function [ a_q, yh, y ] = dpcm_encoder(x, p, qBits, qMin, qMax)

    N = length(x);
    r = zeros(p, 1);
    R = zeros(p, p);

    for i = 1:p
        r(i) = 1/(N - p) * (x(p+1:N)' * x(p+1-i:N-i));
        for j = 1:p
            R(i, j) = 1/(N - p) * (x(p+1-j:N-j)' * x(p+1-i:N-i));
        end
    end

    a_q = kvantistis(R\r, 8, -2, 2);
    y = zeros(N, 1);
    yh = zeros(N, 1);

    y(1:p) = x(1:p);
    yh(1:p) = kvantistis(y(1:p), qBits, qMin, qMax);

    yht = yh(p:-1:1);

    for i = p+1:N
        yt = a_q' * yht;
        y(i) = x(i) - yt;
        yh(i) = kvantistis(y(i), qBits, qMin, qMax);
        yht = [yt + yh(i); yht(1:p-1)];
    end
end
```

dpcm_decoder.m

```
function [ yht ] = dpcm_decoder(a_q, yh)

    N = length(yh);
    p = length(a_q);

    yht = zeros(N, 1);
    yht(1:p) = yh(1:p);

    for i = p+1:N
        yt = a_q' * yht(i-1:-1:i-p);
        yht(i) = yt + yh(i);
    end
end
```

kvantistis.m

```
function [ y_h ] = kvantistis(y, N, min_value, max_value)

    delta = abs(max_value - min_value) / 2^N;

    if y > max_value
        y = max_value;
    elseif y < min_value
        y = min_value;
    end

    y_h = delta * (floor(y/delta) + 0.5);
end
```

Erwthma2.m

```
% Φόρτωση source.mat
load('source.mat');
N = length(t);

% Όρια Κβάντισης
minVal = -3.5;
maxVal = 3.5;

colors = ['-xb'; '-xg'; '-xm'; '-xc'];

% Ερώτηση 2
r=(1:N);
for p = 5:5:10
    figure
    plot(r, t(r), colors(1,:));
    for i = 1:3
        [~, ~, y] = dpcm_encoder(t, p, i, minVal, maxVal);
        hold on
        plot(r, y(r), colors(1+i,:));
    end
    title(['Error comparison p=', num2str(p)]);
    xlabel('Samples'); ylabel('Prediction y');
    legend('x signal', 'y N=1', 'y N=2', 'y N=3');
end
```

Erwthma3.m

```
% Φόρτωση source.mat
load('source.mat');
N = length(t);

% Όρια Κβάντισης
minVal = -3.5;
maxVal = 3.5;

colors = ['-xb'; '-xg'; '-xm'; '-xk'; '-xc'; '-xr'];

% Ερώτηση 3
r=(1:3);
m = zeros(3, 1);
figure
for p = 5:10
    for i = 1:3
        [a_q, ~, y] = dpcm_encoder(t, p, i, minVal, maxVal);
        m(i) = mean(y.^2);
    end
    hold on
    plot(r, m(r), colors(p-4,:));
    disp(['p: ', num2str(p), ', aq: ', num2str(a_q)])
end
```

```

title('Mean squared');
xlabel('N'); ylabel('E(y^2)');
legend('m p=5', 'm p=6', 'm p=7', 'm p=8', 'm p=9', 'm p=10');

```

Erwthma4.m

```

% Φόρτωση source.mat

```

```

load('source.mat');

```

```

N = length(t);

```

```

% Όρια Κβάντισης

```

```

minVal = -3.5;

```

```

maxVal = 3.5;

```

```

colors = ['-xb'; '-xg'; '-xr'; '-xy'];

```

```

% Ερώτηση 4

```

```

r = (1:15);

```

```

for p = 5:5:10

```

```

    figure

```

```

    plot(r, t(r), colors(1,:));

```

```

    for i = 1:3

```

```

        [a_q, yh, ~] = dpcm_encoder(t, p, i, minVal, maxVal);

```

```

        yht = dpcm_decoder(a_q, yh);

```

```

        hold on

```

```

        plot(r, yht(r), colors(1+i,:));

```

```

    end

```

```

    title(['Signal rebuild p=', num2str(p)]);

```

```

    xlabel('Samples'); ylabel('yht');

```

```

    legend('x signal', 'yht N=1', 'yht N=2', 'yht N=3');

```

```

end

```